

Project Workflow: Week 5

Project: EasyStego: A Drag-and-Drop Image Steganography Tool

Phase: Drag-and-Drop Integration

Submitted by: Md. Salauddin | ID: 11240321728

Course: Software Development I (CSE 2216)

Submitted to: Vashkar Kar, Lecturer, Department of CSE, NUBT Khulna

1. Objective

The primary goal of Week 5 was to implement the drag-and-drop functionality that allows users to load images directly into both tabs by dragging them from the operating system file explorer. This included:

- Implementing the custom ImageTransferHandler inner class extending TransferHandler.
- Attaching the TransferHandler to both the Hide Message and Reveal Message drop zones.
- Reading the dropped file as a BufferedImage using javax.imageio.ImageIO.
- Rendering a scaled image preview inside the drop zone after a successful drop.
- Triggering the capacity calculation automatically after an image is loaded in the Hide tab.
- Triggering automatic message extraction after an image is dropped in the Reveal tab.

2. Detailed Task Breakdown

ImageTransferHandler Class: Created a custom inner class ImageTransferHandler extending TransferHandler. The class accepts a Consumer<BufferedImage> callback in its constructor, allowing the same handler to be reused for both the Hide and Reveal drop zones with different actions. The canImport() method checks for DataFlavor.javaFileListFlavor to ensure only file drops are accepted. The importData() method reads the first dropped file as a BufferedImage using ImageIO.read() and passes it to the callback.

Hide Tab Drag-and-Drop: Attached an ImageTransferHandler to hideDropZone with a callback that stores the dropped image in the sourceImage field, calls updateDropZonePreview() to render the image thumbnail, and calls calculateCapacity() to compute and display the maximum embeddable message size in real time.

Reveal Tab Drag-and-Drop: Attached an ImageTransferHandler to revealDropZone with a callback that calls updateDropZonePreview() to render the thumbnail and then calls extractMessage() to automatically decode and display the hidden message. Note: extractMessage() logic will be fully implemented in Week 7; at this stage it is stubbed with a placeholder return.

Image Preview Rendering: Implemented the updateDropZonePreview() helper method. After a successful drop the method clears the drop zone text, scales the BufferedImage to fit within 300x180 pixels using Image.SCALE_SMOOTH, wraps it in an ImageIcon, and sets it as the

drop zone label icon. The dashed border is replaced with a solid 2px accent-color border to indicate a loaded state.

Capacity Calculation: Implemented the `calculateCapacity()` helper method. It reads the width and height of the loaded `BufferedImage`, computes $\text{capacity} = (\text{width} \times \text{height}) / 8$, and updates the `capacityLabel` text to display the result to the user in real time.

Verification: Tested drag-and-drop by dropping both PNG and JPEG images into each tab. Confirmed image previews render correctly, capacity label updates after Hide tab drop, and the Reveal tab callback fires without errors.

3. Key Code — `ImageTransferHandler`

The core drag-and-drop handler implemented this week:

```
class ImageTransferHandler extends TransferHandler {
    private final Consumer<BufferedImage> onImageLoaded;

    public ImageTransferHandler(Consumer<BufferedImage> onImageLoaded)
    {
        this.onImageLoaded = onImageLoaded;
    }

    @Override
    public boolean canImport(TransferSupport support) {
        return
support.isDataFlavorSupported(DataFlavor.javaFileListFlavor);
    }

    @Override
    public boolean importData(TransferSupport support) {
        try {
            List<?> files = (List<?>) support.getTransferable()
                .getTransferData(DataFlavor.javaFileList
Flavor);
            if (!files.isEmpty() && files.get(0) instanceof File) {
                BufferedImage img = ImageIO.read((File) files.get(0));
                if (img != null) { onImageLoaded.accept(img); return
true; }
            }
        } catch (Exception e) { e.printStackTrace(); }
        return false;
    }
}
```

4. Methods Added This Week

Method / Class	Responsibility
ImageTransferHandler	Inner class — handles OS file drops into the application
canImport()	Checks that the drop contains a file (javaFileListFlavor)
importData()	Reads dropped file as BufferedImage and fires the callback
updateDropZonePreview()	Scales image and renders it as a thumbnail inside the drop zone
calculateCapacity()	Computes (width x height) / 8 and updates the capacity label

5. Deliverables

- Updated EasyStego.java: drag-and-drop fully working on both Hide and Reveal tabs.
- ImageTransferHandler inner class implemented and attached to both drop zones.
- updateDropZonePreview() method: scaled thumbnail rendered inside drop zone on drop.
- calculateCapacity() method: real-time capacity display after image load in Hide tab.
- Both PNG and JPEG image drops tested and verified working correctly.
- Updated file uploaded to GitHub.

6. Tools & Resources

Development Environment:

- Java JDK 11+ — compiler and runtime
- Visual Studio Code with Java Extension Pack
- GitHub — file upload and version tracking

Java Classes Used:

- javax.swing.TransferHandler — base class for drag-and-drop handler
- java.awt.datatransfer.DataFlavor — identifies javaFileListFlavor for file drops
- javax.imageio.ImageIO — reads dropped image file into BufferedImage
- java.awt.image.BufferedImage — in-memory image representation
- java.util.function.Consumer<T> — callback to pass loaded image to each tab
- javax.swing.ImageIcon — wraps scaled image for display inside JLabel